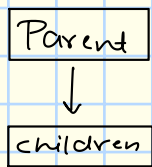


## Principles

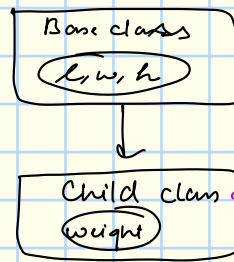
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction



# Inheritance in Java is a fundamental OOPS mechanism that allows a subclass (child class) to acquire the fields & methods of a superclass (parent class).



In oops



child class is inheriting properties from the base class

it will have all property of base class & also additional property

⇒ uses extend class

```
class child extend Base {  
    int weight;  
}
```

you have to initialize variable from base class into child class

⇒ Code parent class & Child class

Box.java

```
package com.anshul.inheritance;  
public class Box { 2 usages 1 inherit  
    double l; 6 usages  
    double h; 6 usages  
    double w; 6 usages  
    Box() { 1 usage  
        this.l = -1;  
        this.h = -1;  
        this.w = -1;  
    }  
  
    // cube  
    Box(double side) { no usages  
        this.l = side;  
        this.h = side;  
        this.w = side;  
    }  
  
    Box(double l, double h, double w) { 1 usage  
        this.l = l;  
        this.h = h;  
        this.w = w;  
    }  
  
    // if passed box inside box means object inside a constructor  
    Box(Box old) { no usages  
        this.l = old.l;  
        this.h = old.h;  
        this.w = old.w;  
    }  
  
    public void info() { no usages  
        System.out.println("Running the box");  
    }  
}
```

iska super bnega  
↳ super();

iska super bnega  
super(side);

super(l, h, w)

Jisme argument wale constructor  
me pass hue hain utne  
me bnega

Boxwt.java

```
package com.anshul.inheritance;  
  
public class Boxwt extends Box { 2 usages  
    double wt; 3 usages  
  
    Boxwt() { no usages  
        this.wt = -1;  
    }  
  
    Boxwt(double l, double h, double w, double wt) { 1 usage  
        super(l, h, w); // what is this?  
        // call the parent class constructor  
        // used to initialise values present in parent class  
        this.wt = wt;  
    }  
}
```

main.java

```
package com.anshul.inheritance;  
  
public class main {  
    public static void main(String[] args) {  
        // Box box = new Box(1.4, 45, 32);  
        // System.out.println(box.l + " " + box.h + " " + box.w);  
  
        // Box box2 = new Box(box);  
        // System.out.println(box.l + " " + box.h + " " + box.w);  
  
        // child class  
        Boxwt b1 = new Boxwt(1.2, 2.3, 4.5, 1.9, 23);  
        System.out.println(b1.l + " " + b1.h + " " + b1.w + " having weight " + b1.wt);  
    }  
}
```

inheritance  
main  
Box  
Boxwt

# private

if made any variable of class private, then only can access in that scope not outside of that.

ex: Assume Box.java file above → make l private.

↳ then we can't access in Boxwt & while making object cannot access that

code

Box.java

```
public class Box {  
    private int l;  
    int h;  
    Box() {  
        this.l = -1;  
        this.h = -1;  
    }  
    Box(int l, int h) {  
        this.l = l;  
        this.h = h;  
    }  
}
```

Boxwt.java

```
public class Boxwt {  
    int wt;  
    Boxwt() {  
        this.wt = -1;  
        this.l = -1;  
    }  
    Boxwt(int l, int h, int wt) {  
        super(l, h);  
        this.wt = wt;  
    }  
}
```

main.java

```
public class main {  
    psvm(...) {  
        Boxwt b1 = new Boxwt(4, 7, 23);  
        cout(b1.l);  
    }  
}
```

"l" has private access in 'com.anshul.inheritance.Box'  
Make 'Box.l' package-private More actions...

but here why no error?

beoz it directly calling from Box function  
that's why not throwing error.

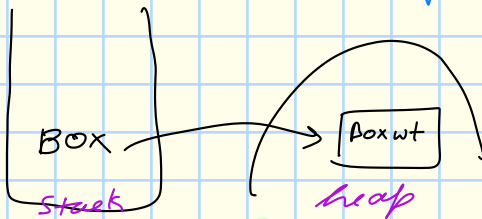
→ Imp.

reference variable. ← Box box = new Boxwt(2, 3, 4, 8);  
cout(box.l);  
cout(box.wt); → It can't access wt

what happening?

- Actually, It is the type of the reference variable (Box box) which is referencing to an object of type (Boxwt)

- It is the reference variable that actually determines, so whatever in (Box) → we can access only those.



⇒ Trying to Reference a child with its parent.

Boxwt box = new Box(2, 3, 4);

↳ error

// there are many variables in both parent & child classes.  
// you are given access to variables that are in the ref type i.e. Boxwt  
// hence, you should have access to weight variable.  
// this also means, that the ones you are trying to access should be initialised  
// but here, when the obj itself is of type parent class, how will you call the constructor of child class.

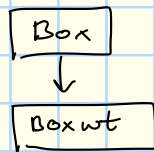
You call constructor of parent class which have don't idea of wt variable.

# "Super" keyword → is a reference variable used inside a subclass to refer directly to its immediate parent class object.

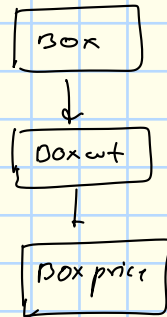
It is primarily used to eliminate ambiguity between subclasses & superclasses that share the same variable or method names, and to handle constructor chaining.

## # Types of Inheritance:

1. Single Inheritance: One class extends another class



2. Multi level Inheritance:



Above classes have no idea of bottom class, but every bottom class have the idea of above classes.

Box.java

Boxwt.java

```
public class Box {
```

```
    int l;
```

```
    int h;
```

```
    Box() {
```

```
        this.l = -1;
```

```
        this.h = -1;
```

```
    }
```

```
    Box(int side) {
```

```
        this.l = side;
```

```
        this.h = side;
```

```
    }
```

```
    Box(int l, int h) {
```

```
        this.l = l;
```

```
        this.h = h;
```

```
    }
```

```
    Box(Box old) {
```

```
        this.l = old.l;
```

```
        this.h = old.h;
```

```
    }
```

```
public class Boxwt extends Box {
```

```
    int wt;
```

```
    Boxwt() {
```

```
        super(); // call constructor from Box which having no attribute
```

```
    }
```

```
    Boxwt(int side, int wt) {
```

```
        super(side);
```

```
        this.wt = wt;
```

```
    }
```

```
    Boxwt(int l, int h, int wt) {
```

```
        super(l, h);
```

```
        this.wt = wt;
```

```
    }
```

```
    Boxwt(Boxwt other) {
```

```
        super(other);
```

```
        this.wt = other.wt;
```

```
    }
```

```
public class Boxprice extends Box
```

```
    int cost;
```

```
    Boxprice() {
```

```
        super();
```

```
        this.cost = -1;
```

```
    }
```

```
    Boxprice(int side, int cost) {
```

```
        super(side);
```

```
        this.cost = cost;
```

```
    }
```

```
    Boxprice(int l, int h, int wt, int cost) {
```

```
        super(l, h, wt);
```

```
        this.cost = cost;
```

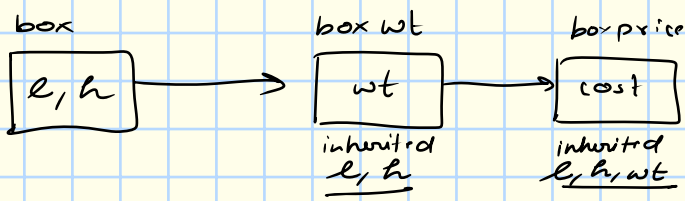
```
    }
```

```
    Boxprice(Boxprice other) {
```

```
        super(other);
```

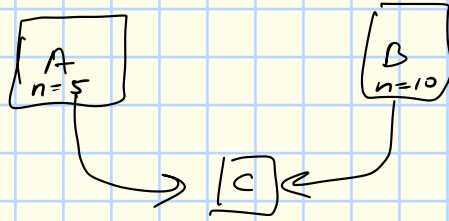
```
        this.cost = other.cost;
```

```
    }
```



3. Multiple Inheritance: One class extending more than 1 classes - **Not** allowed in Java.

ex:-



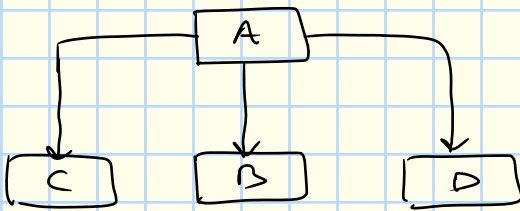
C obj = new C();

C.n → what will it print n=10 or n=5 → **ambiguity**

// Java doesn't support this, As if two or more parent have same variable & property then child class get confuse which one to pick.

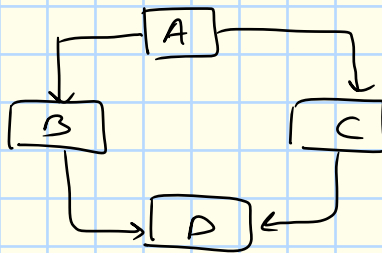
can implement using Interfaces

4. Hierarchical Inheritance One class is inherited by multiple classes.



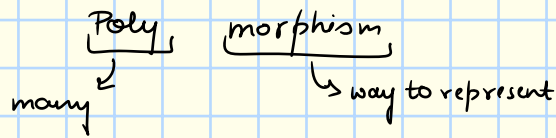
5. Hybrid Inheritance: Combination of single & multiple inheritance

\* Not in java, but can implement using Interfaces.



# Polymorphism

if any language not support this then it not an object oriented language instead they are object basic language

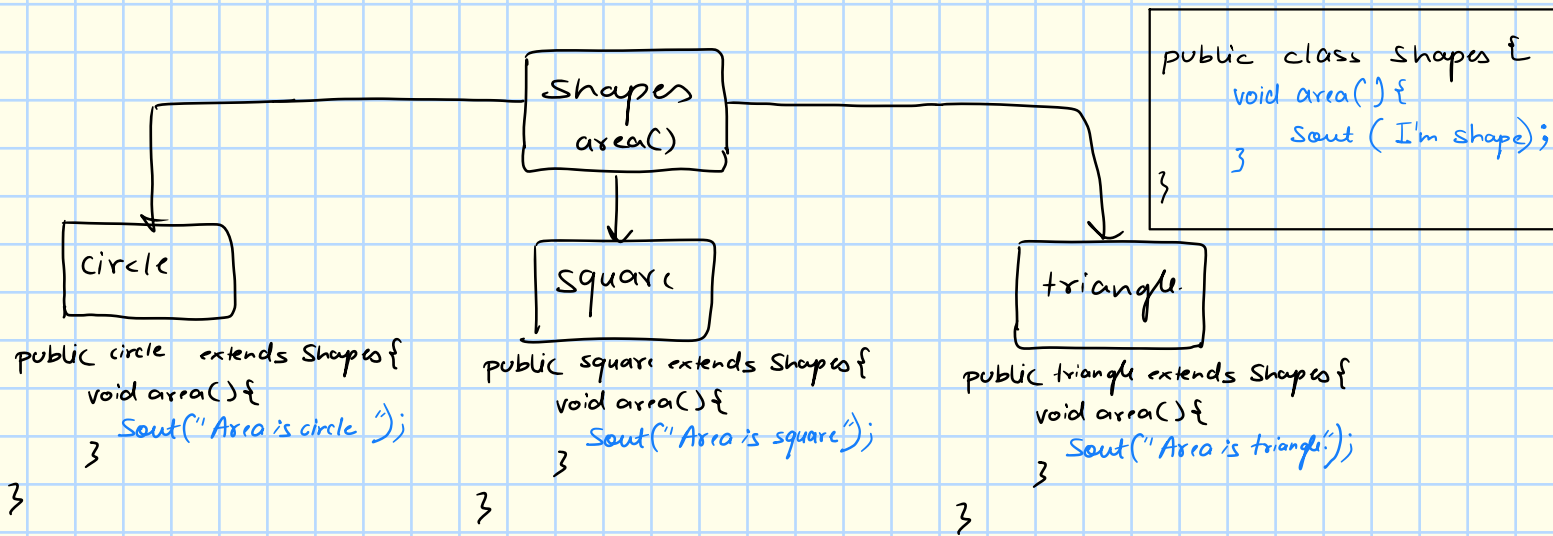


\* Java Support polymorphism.

Polymorphism :- is one of the fundamental concept of (OOP) that allow a single entity to take multiple forms. It enables the same method or interface to behave differently depending on the object involved.

Polymorphism improves code flexibility, reusability, & maintainability.

- It means "many forms"
- It allows the same method to exhibit different behaviour.
- It is achieved through method overloading & method overriding.



```
public class main {  
    psvm (...) {  
        Shapes sl = new Shape();  
        Circle cl = new Circle();  
        Square sql = new Square();  
  
        sql.area(); → "Area is square"  
  
        Shapes square = new Square();  
        square.area(); → "Area is square"  
    }  
}
```

## # Types of Polymorphism.

1) Compile Time / Static Polymorphism → Achieved via method Overloading

Same name but types, argument passed, return types, ordering can be different.

more Ex:-

Order diff. → means

```
A(String a, int b){ }  
A(int b, String a){ }  
A(int a, int b){ }  
A(int b, int a){ }
```

→ overloading

This is not

ex:- multiple constructor.  
A a = new A();  
A a1 = new A(3,4);

⇒ Order of the type should different.

```
public class Num {  
    int sum(int a, int b) {  
        return a+b;  
    }  
    int sum(int a, int b, int c) {  
        return a+b+c;  
    }  
    psum() {  
        Num obj = new Num();  
        obj.sum(3,4);  
        obj.sum(3,4,5);  
        obj.sum(3,4,5,6);  
    }  
}
```

→ error

## 2) Runtime / Dynamic Polymorphism → Achieved via method overriding

Method overriding in Java allows a subclass to provide a specific implementation of a method that is already define in parent class. It is one of the key features of of runtime polymorphism in OOP.

- Access modifier cannot be more restrictive than the parent method
- Achieved when child & parent classes have method with the same name.
- Static, final & private cannot be overridden.

ex:-

```
public class Shapes {  
    void area() {  
        Sout("I'm shape");  
    }  
}  
  
public class Circle extends Shapes {  
    @Override  
    void area() {  
        Sout("Area is circle");  
    }  
}  
  
public class Square extends Shapes {  
    @Override  
    void area() {  
        Sout("Area is square");  
    }  
}  
  
public class Triangle extends Shapes {  
    @Override  
    void area() {  
        Sout("Area is triangle");  
    }  
}
```

whatever the type of object will be that method will be called.

ex:- Shapes circle = new Circle();  
circle.area(); → Area is circle!

↳ will access throw Shapes if area method not present in Shapes then will throw error

Reference type must have area to access area from which object created

ex:- Shapes circle = new Circle();

Reference type → Shapes

obj. created of → Circle.

2) If Reference type is of parent class & object is created of subclass

↳ then Overriding used

```
Parent obj = new Child();
```

Here, which method will be called depends on this is known as **Up casting**

→ Here, Overriding works

Parent obj = new Child()  
which element it can access define this  
which one will actually get access define this.

# How java determine this? (Reference type tells you can access area(), but which particular version of the area to call that is determine by object → but how java determine this?)

```
Shapes circle = new Circle();
```

method check in Shapes. → If available means **area** is accessible bcoz it is available here, Now which one to access that will determine by the java during **run-time** using **dynamic method dispatch**

# Using "final" keyword

↳ can use to prevent overriding, inheritance.  
↳ cannot modify

```
ex:- public class Shapes {  
    final void area() {  
        Sout("I'm shape");  
    }  
}
```

```
public Circle extends Shapes {  
    @Override  
    void area() {  
        Sout("Area is circle");  
    }  
}
```

'area()' cannot override 'area()' in 'com.anshul.Polymorphism.Shapes'; overridden method is final  
Make 'Shapes.area()' not final More actions...

```
final void area() {  
    Sout("I'm shape");  
}
```

→ early binding

final variable → to create constant variable  
final method → Prevent method overriding  
final class → Prevent Inheritance

early binding

1. Compile-time process
2. method definition & method call are linked during the compile time.
3. Actual object is not used for binding
4. example: method overloading
5. Program execution is faster

late binding

1. Run-time process
2. method definition & method call are linked during the run time.
3. Actual object is used for binding
4. example: method overriding
5. Program execution is slower

⇒ Using final to prevent inheritance

↳ make class final and other class cannot inherit that class & it's method  
⇒ When we declare a class final, implicitly it's declare it's method final.

## # Can we override static method?

↳ not get overridden → cannot override them.

ex:-

```
public class Box {  
    static void greeting() {  
    }  
}
```

```
public class Boxwt extends Box {  
    @Override  
    static void greeting() {  
    }  
}
```

Will not get overridden.

**Notes:** Overriding depends on objects, static not depends on objects, hence, static cannot be overridden.

Both overriding & overloading not apply on instance variables.

## # Encapsulation

→ wrapping up the implementation of the data member & methods in a single class.

**Encapsulation in Java** is an object-oriented principle that binds data and methods into a single unit, typically a class. It restricts direct access to data by hiding implementation details. This ensures controlled interaction with the data through defined methods.

- Achieved using access modifiers like private, protected, and public.
- Improves data security by allowing validation through getters and setters.
- Enhances code maintainability by isolating changes within the class.

### How Encapsulation is Achieved in Java

Encapsulation in Java is achieved using:

- Private data members
- Public getter and setter methods

#### Key Rules:

- **Declare data as private:** Hide the class data so it cannot be accessed directly from outside the class.
- **Use getters and setters:** Keep variables private and provide public getter and setter methods for controlled access and safe modification, often with validation.
- **Apply proper access modifiers:** Use private for data hiding and public for methods that provide access.

### Advantages of Encapsulation

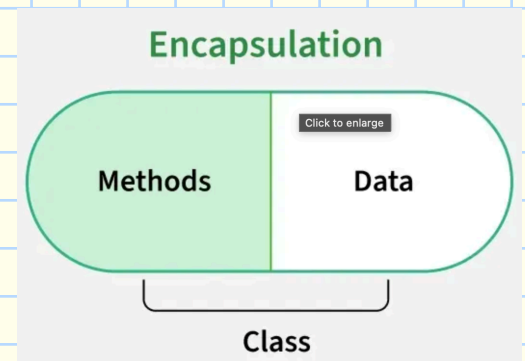
The advantages of encapsulation are listed below:

- **Data Hiding:** Encapsulation restricts direct access to class variables, protecting sensitive data from unauthorized access.
- **Improved Maintainability:** Changes to internal implementation can be made without affecting external code that uses the class.
- **Enhanced Security:** Encapsulation allows validation and control over data, preventing invalid or harmful values from being set.
- **Code Reusability:** Encapsulated classes can be reused in different programs without exposing internal logic.
- **Better Modularity:** Encapsulation promotes organized, modular code by keeping data and methods together within a class.

### Disadvantages of Encapsulation

The disadvantages of encapsulation are listed below:

- **Increased Code Complexity:** Writing getter and setter methods for every variable can make the code longer and slightly more complex.
- **Performance Overhead:** Accessing data through methods instead of directly can introduce a minor performance cost, especially in performance-critical applications.
- **Less Flexibility in Some Cases:** Over-restricting access to class members may limit the ability of other classes to extend or use the class efficiently.



### Data Hiding vs Encapsulation

| Aspect             | Data Hiding                                                           | Encapsulation                                                                  |
|--------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Definition         | Restricting direct access to data to protect it from unauthorized use | Wrapping data and methods into a single unit (class)                           |
| Focus              | Security of data                                                      | Structure and design                                                           |
| Purpose            | Prevents misuse and accidental modification of data                   | Improves modularity and maintainability                                        |
| How it is achieved | Using access modifiers (private, protected)                           | Using classes, methods, and access modifiers                                   |
| Involves methods   | No (mainly focuses on data)                                           | Yes (includes both data and methods)                                           |
| Scope              | Narrow concept                                                        | Broader concept                                                                |
| Dependency         | Independent concept                                                   | Encapsulation may use data hiding for better security but it is not mandatory. |
| Example            | private int balance;                                                  | class Account { private int balance; public getBalance(); }                    |

# # Abstraction

- Hiding unnecessary details
- Sharing valuable information.

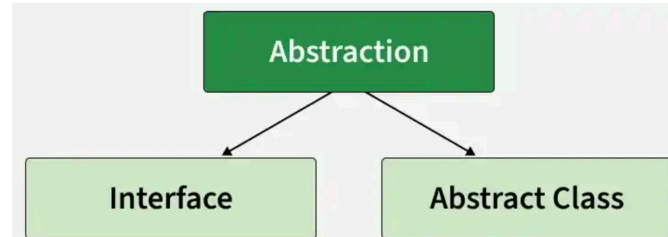
**Abstraction in Java** is the process of hiding internal implementation details and showing only essential functionality to the user. It focuses on what an object does rather than how it does it.

- It hides the complex details and shows only essential features.
- Abstract classes may have methods without implementation and must be implemented by subclasses.
- By abstracting functionality, changes in the implementation do not affect the code that depends on the abstraction.

## Real-Life Example of Abstraction

The television remote control is the best example of abstraction. It simplifies the interaction with a TV by hiding all the complex technology. We don't need to understand how the TV internally works; we just need to press the button to change the channel or adjust the volume.

## Ways to Achieve Abstraction



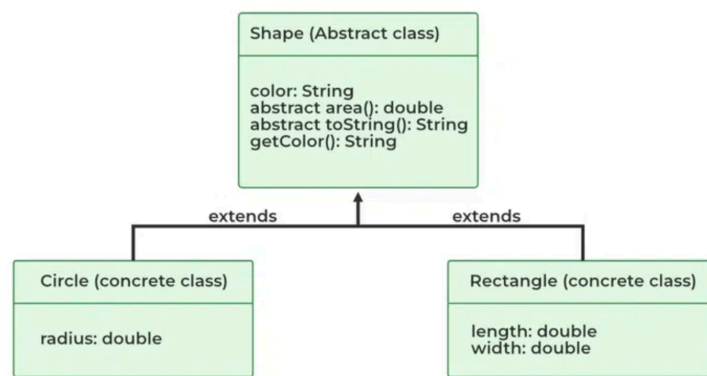
Java provides two ways to implement abstraction, which are listed below:

- Abstract Classes (Partial Abstraction)
- Interface (provides abstraction for behavior, may contain default or static methods)

## Abstract class

An **abstract class** is a class that cannot be instantiated and may contain abstract methods (methods without body) as well as concrete methods (with implementation).

In shape example, a base type Shape has common properties like color and size. Specific shapes—Circle, Square, Triangle—inherit from Shape and may have unique properties or behaviors. For instance, some shapes can be flipped, and each shape calculates its area differently, illustrating both shared features and individual differences.



Abstract Classes and methods1yh

**Example:** This program demonstrates partial abstraction by defining an abstract class Shape that includes an abstract method area(), a concrete method getColor(), and a constructor.

## Interface

An **interface** is a blueprint for a class that defines a set of methods a class must implement. It is commonly used to achieve abstraction in Java. Modern Java interfaces can contain abstract methods, constants, and also default or static methods with implementations.

## Advantages of Abstraction

- Abstraction makes complex systems easier to understand by hiding the implementation details.
- Abstraction keeps different part of the system separated.
- Abstraction maintains code more efficiently.
- Abstraction increases the security by only showing the necessary details to the user.

## Disadvantages of Abstraction

- It can add unnecessary complexity if overused.
- May reduce flexibility in implementation.
- Makes debugging and understanding the system harder for unfamiliar users.
- Overuse of abstraction may make code harder to follow, though performance impact is usually negligible.

## Common Mistakes to Avoid

The common mistakes that can occur and we should avoid when working with Abstraction in Java are listed below:

- **Not Implementing Abstract Methods:** Always make sure that the abstract methods are implemented in the concrete subclass.
- **Overusing Abstraction:** Avoid making everything abstract when it's not required. Use abstraction only when it enhances the design.
- **Inconsistent Method Signatures in Subclasses:** When you override abstract methods, please make sure the method signature matches exactly, any mistake can cause errors.

# # Abstraction V/s Encapsulation

## Abstraction vs Encapsulation

| Feature        | Abstraction                                                                                             | Encapsulation                                                                                            |
|----------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Definition     | Abstraction is the process of hiding implementation details and showing only functionality to the user. | Encapsulation is the process of binding data and methods together and restricting direct access to data. |
| Focus          | Abstraction focuses on what an object does rather than how it does it.                                  | Encapsulation focuses on protecting data by controlling how it is accessed.                              |
| Goal           | Abstraction helps reduce complexity and improve code maintainability.                                   | Encapsulation helps ensure data security and maintain data integrity.                                    |
| Level          | Abstraction works at the design or interface level.                                                     | Encapsulation works at the implementation level.                                                         |
| Implementation | Abstraction is implemented using abstract classes and interfaces.                                       | Encapsulation is implemented using access modifiers like private, public, and protected.                 |
| Data Handling  | Abstraction exposes only the necessary features to the user.                                            | Encapsulation hides data and provides controlled access using getters and setters.                       |

